PROJECT DOCUMENTATION

Animated Presentation on

Machine Learning Algorithms

NAME- KAZI EAJAJ HOSSAIN

STUDENT CODE- BWU/BTA/22/335

Project Name	Animated Presentation on ML Algorithms
Technology Stack	Python · Streamlit · Plotly · Scikit-learn
Platform	Windows (Web Browser via localhost)
Version	1.0.0
Year	2025–2026
Document Type	Full Project Documentation

1. Project Overview

This document provides comprehensive documentation for the Animated Presentation on Machine Learning Algorithms — an interactive Streamlit web application designed to teach core machine learning concepts through live visualizations, adjustable demos, and real-time algorithm benchmarks.

The application is built entirely in Python and runs locally on Windows via a web browser. It covers six fundamental ML algorithms across three learning paradigms: supervised learning, unsupervised learning, and ensemble/deep learning methods.

> Project Name: Animated Presentation on Machine Learning Algorithms
> Type: Interactive Web Application (Streamlit)
> Platform: Windows 10 / 11 | Python 3.9+
> Libraries: Streamlit, Plotly, Scikit-learn, NumPy, Pandas
> Purpose: Education — ML algorithm visualization & demonstration

1.1 Motivation

Machine learning algorithms are often taught through static textbooks and diagrams, which makes it difficult to build intuitive understanding of dynamic processes such as gradient descent convergence, centroid movement in K-Means, or how a neural network's decision boundary evolves with training. This project addresses that gap by creating a fully interactive educational tool.

1.2 Objectives

- Provide animated, interactive visualizations for 6 major ML algorithms.
- Allow users to control algorithm parameters via sliders and dropdowns and see results in real time.
- Run live algorithm benchmarks on synthetic datasets inside the browser.
- Serve as an educational resource for students, educators, and beginners in data science.
- Be easy to install and run on any Windows machine with Python installed.

1.3 Scope

The application covers six core ML algorithm families. Supervised learning is represented by Linear Regression, Decision Trees, and Support Vector Machines. Unsupervised learning is covered by K-Means Clustering. The deep learning section focuses on Feedforward Neural Networks (MLP), and the ensemble section covers Random Forest. A dedicated comparison page benchmarks all classifiers simultaneously on user-configurable synthetic data.

2. System Requirements

2.1 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 or higher
RAM	4 GB	8 GB or more
Storage	500 MB free space	1 GB free space
Display	1280 × 720	1920 × 1080 (Full HD)
Internet	Required (first install only)	Broadband (faster install)

2.2 Software Requirements

Software	Version	Purpose
Windows	10 / 11 (64-bit)	Operating System
Python	3.9 or higher	Runtime environment
pip	Latest (bundled with Python)	Package manager
Web Browser	Chrome / Edge / Firefox	View the Streamlit app
Streamlit	1.32.0+	Web app framework
Plotly	5.18.0+	Interactive charts
Scikit-learn	1.4.0+	ML algorithms
NumPy	1.26.0+	Numerical computing
Pandas	2.0.0+	Data structures

> NOTE: Python must be added to the system PATH during installation.
> When installing Python, check the box that says 'Add Python to PATH'.
> Without this, commands like 'python' and 'pip' will not work in Command Prompt.

3. Project Structure

The project follows a modular, page-based architecture. Each algorithm has its own Python module inside the pages/ directory. The main app.py file handles routing and shared styling.

3.1 Folder Layout

```
ml_Algorithm_streamlit_app/
    app.py                  ← Main entry point (run this file)
    requirements.txt        ← Python package dependencies
    setup_and_run.bat       ← Windows one-click launcher
    README.md               ← Quick-start instructions
    pages/
        __init__.py         ← Makes pages a Python package
        home.py             ← Home page: overview & radar chart
        linear.py           ← Linear Regression module
        decision.py         ← Decision Trees module
        kmeans.py           ← K-Means Clustering module
        svm.py              ← Support Vector Machine module
        neural.py           ← Neural Networks module
        forest.py           ← Random Forest module
        compare.py          ← Algorithm Comparison & Benchmark
```

3.2 File Descriptions

app.py — Main Application

This is the entry point of the entire application. It configures the Streamlit page (title, icon, layout), injects the global CSS styling (custom fonts, color variables, card styles, table styles), renders the sidebar navigation menu, and routes the user to the correct page module based on their selection. All navigation state is managed through st.session_state.

pages/home.py — Home Dashboard

Displays a hero banner introducing the project, four metric cards (algorithms covered, ML categories, charts, demos), a grid of algorithm summary cards with color-coded accent bars, and a Plotly radar chart comparing all six algorithms across five capability dimensions: Accuracy, Speed, Interpretability, Scalability, and Non-Linear handling.

pages/linear.py — Linear Regression

Three-tab layout. The Concept tab explains the algorithm, formula ($y = mx + b$), cost function (MSE), and use cases. The Animated Demo tab features a slider that manually steps through gradient descent iterations, showing how the regression line converges toward the optimal fit while displaying MSE in real time. The Live Experiment tab lets users control data points, noise level, and true slope, then fits a model using scikit-learn and shows the R^2 score.

pages/decision.py — Decision Trees

Explains Gini impurity and Information Gain. The animated demo renders the decision boundary for a 2D classification dataset as the user adjusts max tree depth, clearly showing how deeper trees overfit. The experiment tab lets users toggle the split criterion (Gini vs Entropy) and view tree node count.

pages/kmeans.py — K-Means Clustering

Implements a manual K-Means loop (without scikit-learn) so iteration-by-iteration centroid movement can be animated via sliders. Uses star markers for centroids and circle markers for data points. The experiment tab additionally plots the Elbow Method chart to guide K selection.

pages/svm.py — Support Vector Machine

Visualizes the decision boundary, margin lines (dashed), and support vectors (highlighted with rings) for both linearly separable and circular datasets. Supports three kernels: linear, RBF, and polynomial. Users can adjust the C parameter to see the trade-off between margin width and misclassifications.

pages/neural.py — Neural Networks

The Architecture tab renders a fully dynamic network diagram using Plotly — nodes and edges scale with user-selected hidden layer count and neuron count, with edge opacity reflecting random weight magnitudes. The Live Training tab trains a scikit-learn MLPClassifier on moon-shaped data and renders the loss curve alongside the decision boundary.

pages/forest.py — Random Forest

Compares a single Decision Tree against a Random Forest using 5-fold cross-validation, rendered as a bar chart. Plots accuracy vs number of trees showing convergence, and a feature importance horizontal bar chart. Uses scikit-learn's RandomForestClassifier with configurable n_estimators.

pages/compare.py — Algorithm Comparison

Shows a static HTML comparison table of all algorithms across interpretability, scalability, non-linear capability, and best use case. A live benchmark button trains five classifiers simultaneously on user-configured synthetic data, runs 5-fold cross-validation, and displays a ranked bar chart plus a multi-axis radar chart.

4. Installation & Setup Guide (Windows)

Follow these steps exactly to install and run the application on a Windows computer. The entire process takes approximately 5–10 minutes depending on your internet speed.

4.1 Step 1 — Install Python

1. Open your browser and go to: https://www.python.org/downloads/
2. Download the latest Python 3.x installer for Windows.
3. Run the installer. On the first screen, check the box: 'Add Python to PATH'.
4. Click 'Install Now' and wait for the installation to complete.
5. Open Command Prompt (press Win + R, type cmd, press Enter).
6. Verify installation by typing: python --version

> Expected output: Python 3.11.x (or similar version >= 3.9)
> If you see an error, Python was not added to PATH. Re-run the installer and check the PATH box.

4.2 Step 2 — Extract the Project

7. Download the project ZIP file: ML_Algorithms_Streamlit_App.zip
8. Right-click the ZIP file → 'Extract All...'
9. Choose a destination folder (e.g., C:\Users\YourName\Desktop\ml_project)
10. Click 'Extract'.

4.3 Step 3 — Run the App

Option A: One-Click Launcher (Recommended)

11. Open the extracted ml_project folder.
12. Double-click the file: setup_and_run.bat
13. A Command Prompt window will open and automatically:

- Create a Python virtual environment (venv/)
- Activate the virtual environment
- Install all required packages from requirements.txt
- Launch the Streamlit app

14. Your browser will open automatically at http://localhost:8501

> The first run takes 2–5 minutes to download and install packages.
> Subsequent runs are instant because packages are already cached in venv/.

Option B: Manual Setup via Command Prompt

Use this method if the .bat file does not work on your system.

```
# Open Command Prompt in the ml_project folder:
```

```
# (Shift + Right-click inside the folder → 'Open PowerShell/Command Prompt here')

# Create virtual environment
python -m venv venv

# Activate the virtual environment
venv\Scripts\activate

# Install all dependencies
pip install -r requirements.txt

# Launch the Streamlit application
streamlit run app.py
```

4.4 Step 4 — Access the App

After the app starts, open your browser and navigate to:

http://localhost:8501

The application will load in your browser. Use the left sidebar to navigate between algorithm pages.

> To stop the app: press Ctrl + C in the Command Prompt window.
> To restart: run 'streamlit run app.py' again (virtual environment stays active).
> The app only works while the Command Prompt window is open.

5. Application Features

Every algorithm page in the application is organized into three interactive tabs, ensuring a consistent and progressive learning experience from theory to practice.

5.1 Tab Structure (All Algorithm Pages)

Tab	Name	Contents
Tab 1	Concept	Algorithm explanation, mathematical formulas, step-by-step breakdown, pros & cons, real-world use cases
Tab 2	Animated Demo	Interactive slider or control to animate the algorithm step-by-step with live Plotly charts
Tab 3	Live Experiment	Full scikit-learn implementation with adjustable parameters, real-time accuracy metrics, and additional charts

5.2 Feature Summary by Page

Page	Key Interactive Features
Home	Algorithm radar chart (Plotly), metric cards, algorithm card grid with color-coded accents
Linear Regression	Gradient descent step slider (animated line convergence), MSE display, R^2 metric, noise & slope controls
Decision Trees	Max depth slider → live decision boundary, accuracy readout, Gini vs Entropy selector, node count metric
K-Means	Iteration slider (manual centroid animation), K selector, Elbow Method chart, inertia metric
SVM	Kernel selector (linear/RBF/poly), C parameter slider, margin visualization, support vector highlighting
Neural Networks	Architecture builder (hidden layers + neuron count), live training on moon data, loss curve chart
Random Forest	Single tree vs forest CV accuracy bar chart, accuracy-vs-trees line chart, feature importance bar chart
Comparison	Static capability table, live 5-fold CV benchmark of 5 algorithms, radar comparison chart

6. Machine Learning Algorithms Covered

6.1 Linear Regression

Linear Regression is the simplest supervised learning algorithm. It models the relationship between input features and a continuous output variable by fitting a straight line (or hyperplane in higher dimensions) through the training data. The optimal parameters are found by minimizing the Mean Squared Error (MSE) using Gradient Descent.

```
Formula: y = β₀ + β₁x₁ + ... + βₙxₙ      Cost: J(θ) = (1/2m) Σ (ŷᵢ − yᵢ)²
```

- Type: Supervised Learning (Regression)
- Scikit-learn class: LinearRegression()
- Key parameters: fit_intercept, normalize
- Use cases: House price prediction, sales forecasting, salary estimation

6.2 Decision Trees

A Decision Tree recursively partitions the feature space into rectangular regions by selecting the feature and threshold that maximizes information gain at each node. Leaves are assigned the majority class (classification) or mean value (regression) of training samples that fall in that region.

- Type: Supervised Learning (Classification & Regression)
- Scikit-learn class: DecisionTreeClassifier()
- Key parameters: max_depth, criterion (gini/entropy), min_samples_split
- Use cases: Medical diagnosis, customer segmentation, fraud detection

6.3 K-Means Clustering

K-Means is an iterative unsupervised algorithm that partitions data into K clusters by alternating between two steps: assigning each data point to its nearest centroid, then recomputing each centroid as the mean of its assigned points. It converges when centroid positions stabilize.

- Type: Unsupervised Learning (Clustering)
- Scikit-learn class: KMeans()
- Key parameters: n_clusters, init, n_init, max_iter
- Use cases: Customer segmentation, document clustering, image compression

6.4 Support Vector Machine (SVM)

SVM finds the hyperplane that maximizes the margin between two classes. The Kernel Trick (via the kernel parameter) implicitly maps input features into a higher-dimensional space, enabling non-linear classification without explicitly computing the transformation.

- Type: Supervised Learning (Classification & Regression)
- Scikit-learn class: SVC()
- Key parameters: C, kernel (linear/rbf/poly), gamma, degree
- Use cases: Image classification, text categorization, bioinformatics

6.5 Neural Networks (MLP)

A Multi-Layer Perceptron (MLP) consists of an input layer, one or more hidden layers, and an output layer. Each neuron computes a weighted sum of its inputs and applies a non-linear activation function (ReLU, sigmoid, etc.). The network learns via backpropagation — computing gradients of the loss with respect to all weights and updating them using gradient descent.

- Type: Deep Learning (Classification & Regression)
- Scikit-learn class: MLPClassifier()
- Key parameters: hidden_layer_sizes, activation, learning_rate_init, max_iter
- Use cases: Image recognition, NLP, speech synthesis, anomaly detection

6.6 Random Forest

Random Forest is an ensemble method that builds many Decision Trees, each trained on a random bootstrap sample of the data and using a random subset of features at each split. Final predictions are made by majority vote (classification) or averaging (regression). This approach reduces variance and prevents overfitting compared to a single tree.

- Type: Ensemble Learning (Classification & Regression)
- Scikit-learn class: RandomForestClassifier()
- Key parameters: n_estimators, max_depth, max_features, bootstrap
- Use cases: General-purpose classification, feature selection, medical outcomes

7. Code Architecture & Design Decisions

7.1 Framework: Streamlit

Streamlit was chosen as the web framework because it allows data scientists to build interactive web applications entirely in Python with no knowledge of HTML, CSS, or JavaScript. Widgets like sliders, selectboxes, and buttons are declared as simple function calls and automatically trigger re-execution of the script, making real-time interactivity trivial to implement.

7.2 Charting: Plotly

All visualizations use Plotly's graph_objects API for maximum control over chart appearance. Plotly charts are interactive by default — users can hover, zoom, pan, and toggle legend items. Custom color schemes, background colors, and font settings are applied to every chart to maintain a consistent dark-themed aesthetic matching the app's CSS.

7.3 Navigation Pattern

Rather than using Streamlit's built-in multi-page file convention (which requires separate files at the top level), the app uses a custom session-state router. The sidebar renders buttons for each page; clicking a button sets st.session_state.page to the chosen key and calls st.rerun(). The main script then imports and calls the render() function of the selected page module.

> Advantage: All pages share the same CSS injection (from app.py), eliminating the need to repeat styling code in every file. It also allows full control over the sidebar appearance.

7.4 CSS Theming

A comprehensive custom CSS block is injected via st.markdown(..., unsafe_allow_html=True) in app.py. It defines CSS variables (--navy, --blue, --accent, etc.) and utility classes (.algo-card, .step-box, .formula-box, .metric-card, .section-title, .cmp-table) that are reused across all pages. This ensures visual consistency without repeating style declarations.

7.5 Reproducibility

All random number generators are seeded with fixed values (np.random.seed(42), random_state=0, etc.) so that charts always render identically. This prevents confusion when learners compare their results to documentation or screenshots.

7.6 Virtual Environment

The setup_and_run.bat script creates an isolated Python virtual environment (venv/) inside the project folder. This prevents dependency conflicts with other Python projects on the user's machine and ensures the exact library versions specified in requirements.txt are used.

8. Usage Guide

8.1 Navigating the App

- The left sidebar lists all 8 pages. Click any item to navigate instantly.
- The active page is highlighted in the sidebar with a blue accent.
- All pages load pre-computed visualizations immediately; no manual refresh is needed.
- Changing any slider or dropdown instantly re-renders the affected chart.

8.2 Using the Animated Demos

On each algorithm's 'Animated Demo' tab, a slider controls the progression of the animation. Drag it slowly from left to right to watch the algorithm evolve step by step. The chart title updates to show current parameter values and metrics (e.g., MSE, accuracy, iteration number).

8.3 Running the Live Benchmark

15. Navigate to the 'Algorithm Comparison' page using the sidebar.
16. Adjust the 'Dataset Size' and 'Number of Features' sliders.
17. Click the 'Run Benchmark' button.
18. Wait 10–20 seconds while all five classifiers train and evaluate.
19. View the ranked bar chart and radar comparison chart.

> The benchmark uses 5-fold cross-validation, which trains each model 5 times.
> Larger datasets and more features will take longer to compute.
> Error bars on the bar chart show standard deviation across folds.

8.4 Keyboard Shortcuts

Shortcut	Action
Ctrl + C (in terminal)	Stop the Streamlit server
F5 (in browser)	Refresh / restart the page
Ctrl + Shift + L	Clear Streamlit cache (if app seems stuck)
Arrow keys on slider	Fine-tune slider values one step at a time

9. Troubleshooting

9.1 Common Issues & Solutions

Problem	Solution
'python' is not recognized	Python not in PATH. Re-install Python and check 'Add to PATH' box.
'streamlit' is not recognized	Virtual environment not activated. Run: venv\Scripts\activate
ModuleNotFoundError: No module named 'plotly'	Run: pip install -r requirements.txt inside the project folder.
Browser doesn't open automatically	Manually open: http://localhost:8501 in your browser.
Port 8501 already in use	Run: streamlit run app.py --server.port 8502 to use a different port.
Charts not rendering / blank page	Press F5 in browser. If persistent, stop and restart the server.
App very slow on benchmark	Reduce dataset size slider. 5-fold CV on large data takes longer.
setup_and_run.bat blocked by Windows	Right-click the .bat file → 'Run as Administrator'

9.2 Reinstalling Dependencies

If packages are corrupted or mismatched, delete the venv/ folder and re-run the bat file, or manually run:

```
# Delete virtual environment
rmdir /s /q venv

# Recreate and reinstall
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
streamlit run app.py
```

10. Technology Stack

Library	Version	Role in Project
Streamlit	1.32+	Web framework — UI components, state, layout, navigation
Plotly	5.18+	All interactive charts — scatter, contour, bar, radar, line
Scikit-learn	1.4+	All ML algorithm implementations and cross-validation
NumPy	1.26+	Numerical arrays, random data generation, meshgrid for boundaries
Pandas	2.0+	Data structures (used internally by Streamlit and Plotly)
Python	3.9+	Core programming language and runtime

11. References

The following resources informed the design, content, and implementation of this project:

20. Mitchell, T. M. (1997). Machine Learning. McGraw-Hill Education.
21. Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.
22. Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow (3rd ed.). O'Reilly Media.
23. Scikit-learn Documentation. https://scikit-learn.org/stable/
24. Streamlit Documentation. https://docs.streamlit.io/
25. Plotly Python Documentation. https://plotly.com/python/
26. Mayer, R. E. & Moreno, R. (2003). Nine ways to reduce cognitive load in multimedia learning. Educational Psychologist, 38(1), 43–52.